

Redis issue-3221 분석

1 개요

redis server에 BITFIELD 명령을 보내면 크래시 되는 이슈

1.1 재현 명령

```
SET foo 1
BITFIELD foo GET i1 0      # value foo의 첫번째 비트를 signed integer의 형태로 읽음
```

2 로그

```
... # 생략
2093137:M 27 Sep 15:48:26.933 # Server started, Redis version 999.999.999
2093137:M 27 Sep 15:48:26.933 # WARNING you have Transparent Huge Pages (THP)
support enabled in your kernel. This will create latency and memory usage issues
with Redis. To fix this issue run the command 'echo never >
/sys/kernel/mm/transparent_hugepage/enabled' as root, and add it to your
/etc/rc.local in order to retain the setting after a reboot. Redis must be
restarted after THP is disabled.
2093137:M 27 Sep 15:48:26.933 * The server is now ready to accept connections on
port 6379
2093137:M 27 Sep 15:48:26.934 - 0 clients connected (0 slaves), 762624 bytes in use
2093137:M 27 Sep 15:48:31.953 - 0 clients connected (0 slaves), 762624 bytes in use
2093137:M 27 Sep 15:48:33.216 - Accepted 127.0.0.1:58436

... # 크래시 발생

=== REDIS BUG REPORT START: Cut & paste starting from here ===
2093137:M 27 Sep 15:48:36.145 # Redis 999.999.999 crashed by signal: 11
2093137:M 27 Sep 15:48:36.145 # Crashed running the instuction at: 0x47d221
2093137:M 27 Sep 15:48:36.145 # Accessing address: (nil)
2093137:M 27 Sep 15:48:36.145 # Failed assertion: <no assertion failed> (<no
file>:0)

----- STACK TRACE -----
EIP:
./redis/src/redis-server *:6379(bitfieldCommand+0x3f1)[0x47d221]

Backtrace:
./redis/src/redis-server *:6379(logStackTrace+0x6b)[0x4670fb]
```

```
./redis/src/redis-server *:6379(sigsegvHandler+0xaf)[0x46766f]
/nix/store/fg4yq8i8wd08xg3fy58l6q73cgy8hjr2-glibc-2.27/lib/
libpthread.so.0(+0x11f10)[0x75b151c11f10]
./redis/src/redis-server *:6379(bitfieldCommand+0x3f1)[0x47d221]
./redis/src/redis-server *:6379(call+0x71)[0x42ac81]
./redis/src/redis-server *:6379(processCommand+0x45f)[0x42df8f]
./redis/src/redis-server *:6379(processInputBuffer+0x115)[0x43b155]
./redis/src/redis-server *:6379(aeProcessEvents+0x124)[0x424b04]
./redis/src/redis-server *:6379(aeMain+0x2b)[0x424ebb]
./redis/src/redis-server *:6379(main+0x200)[0x421b30]
/nix/store/fg4yq8i8wd08xg3fy58l6q73cgy8hjr2-glibc-2.27/lib/
libc.so.6(__libc_start_main+0xee)[0x75b151821b8e]
./redis/src/redis-server *:6379(_start+0x2a)[0x421fda]
... # 생략
```

redis의 버그 리포트에서 bitfieldCommand() 함수를 발견

3 원인

redis의 value는 기본적으로 string으로 저장되지만 long형태의 정수로 표현될 수 있는 경우에는 공간 절약을 위해 long의 형태로 저장

bitfieldCommand() 함수는 모든 value가 string으로 저장되었다고 가정하기 때문에 문제 발생

3.1 과정

3.1.1

SET foo 1 명령을 통해 redis로 값을 저장할 때 tryObjectEncoding() 함수를 통해 string을 long으로 변환 후 저장하는 과정을 거침

3.1.1.1 string 형태의 value를 long 형태로 다시 저장하는 코드

```
/* Try to encode a string object in order to save space */
robj *tryObjectEncoding(robj *o) {
    long value;
    sds s = o->ptr;
    size_t len;

... # 생략

    /* Check if we can represent this string as a long integer.
     * Note that we are sure that a string larger than 21 chars is not
     * representable as a 32 nor 64 bit integer. */
    len = sdslen(s);
    if (len <= 21 && string2l(s, len, &value)) {
... # 생략

        if (o->encoding == OBJ_ENCODING_RAW) sdsfree(o->ptr);
        o->encoding = OBJ_ENCODING_INT;
        o->ptr = (void*) value;
        return o;

... # 생략
```

string2l() 함수로 string을 long으로 변환하여 value에 정수값 1 저장

이를 o->ptr에 void *로 캐스팅해 전달

3.1.2

BITFIELD GET 명령 수행 중 o->ptr의 내용을 string으로 잘못 알고 진행 중 크래시

3.1.2.1 BITFIELD 명령을 수행하는 코드

```
void bitfieldCommand(client *c) {
    robj *o;
... # 생략

    /* GET */
    o = lookupKeyRead(c->db,c->argv[1]);
    size_t olen = (o == NULL) ? 0 : sdslen(o->ptr);

... # 생략
```

sdslen는 주어진 string의 길이를 구하는 함수

o->ptr의 값은 1이기 때문에 sdslen(o->ptr)에서 잘못된 메모리 주소 접근에 의한 segmentation fault

4 수정

```
void bitfieldCommand(client *c) {
    robj *o;
... # 생략

    /* GET */
-    o = lookupKeyRead(c->db,c->argv[1]);
-    size_t olen = (o == NULL) ? 0 : sdslen(o->ptr);
    unsigned char buf[9];
+    size_t olen = 0;
+    unsigned char *src = NULL;
+    char llbuf[32];

+    o = lookupKeyRead(c->db,c->argv[1]);

+    /* Set the 'p' pointer to the string, that can be just a stack
+    allocated
+    * array if our string was integer encoded. */
+    if (o && o->encoding == OBJ_ENCODING_INT) {
+        src = (unsigned char*) llbuf;
+        olen = ll2string(llbuf,sizeof(llbuf),(long)o->ptr);
+    } else if (o) {
+        src = (unsigned char*) o->ptr;
+        olen = sdslen(o->ptr);
+    }
}
```

```
/* For GET we use a trick: before executing the operation
 * copy up to 9 bytes to a local buffer, so that we can easily
 * execute up to 64 bit operations that are at actual string
 * object boundaries. */
```

```
memset(buf,0,9);
```

```
- unsigned char *src = o ? o->ptr : NULL;
```

o->encoding이 OBJ_ENCODING_INT인 경우를 따로 처리하여 해결